

Protokoll: Prismenspektrometer
Teilnehmer: Florian Hildebrandt, Merlin Gromer
Datum: 23. 04. 2026

Zusammenfassung der Aufgabe.....	2
Verwendete Geräte:.....	3
Durchführung.....	3
3.1: Arretierung.....	3
3.2: Einstellung der Spaltbreite.....	4
3.3: Bestimmung des Prismenwinkels γ und der Nullrichtung.....	5
3.4.1 & 4.1.1 Messung des minimalen Ablenkungswinkels (Links).....	6
3.4.2 & 4.1.2 Messwerte (spiegelbildliche Ablenkung) (Rechts).....	7
4.2 Tabelle λ & Ablenkungswinkel $\delta_{\min}$	8
4.3 Kalibrierkurve: $\delta_{\min}(\lambda)$	9
4.4.1 Dispersionskurve $n(\lambda)$ und Cauchy-Fit.....	10
4.4.2 Glas Bestimmen:.....	11
4.5 Fehler der Brechungsindices.....	12
4.6 Gleichung 4 Herleiten.....	14
Anhang.....	15
Messwerte Protokoll.....	15
Python: Kalibrierkurve (auch im GitHub Repository):	18
Python: Dispersionskurve & Cauchy (GitHub)	19
Python: Fehlerfortpflanzung & Brechungsindices.....	20

Zusammenfassung der Aufgabe

Durch Messungen mit einem Prismenspektrometer wird die wellenlängenabhängige Brechzahl $n(\lambda)$ eines Prismenglases bestimmt sowie die Emissionswellenlängen verschiedener Lichtquellen (z. B. Heliumlampe und LEDs) untersucht. Dabei wird der Zusammenhang zwischen Ablenkwinkel des Lichts im Prisma und der Wellenlänge analysiert.

Methode 1:

Zunächst wird der Prismenwinkel γ bestimmt, indem die Winkel der reflektierten Strahlen an den Prismenflächen gemessen werden. Anschließend wird ohne Prisma die Nullrichtung des Systems aufgenommen.

Danach wird für verschiedene Spektrallinien der Winkel der minimalen Ablenkung δ_{min} gemessen.

Mit Hilfe der Beziehung zwischen Prismenwinkel, minimalem Ablenkwinkel und Brechungsindex wird daraus für jede Wellenlänge die Brechzahl $n(\lambda)$ berechnet.

Es werden außerdem die zugehörigen Messunsicherheiten bestimmt.

Methode 2:

Die Messungen der minimalen Ablenkung werden für mehrere bekannte Wellenlängen (z. B. aus dem Heliumspektrum) durchgeführt. Aus den gemittelten Ablenk winkeln wird eine Kalibrierkurve $\delta_{min(\lambda)}$ erstellt.

Damit lassen sich unbekannte Wellenlängen (z. B. von LEDs) bestimmen. Gleichzeitig wird erneut der Brechungsindex berechnet und mit den vorherigen Ergebnissen verglichen.

Methode 3:

Die berechneten Brechungsindizes werden als Funktion der Wellenlänge dargestellt. An diese Messdaten wird mithilfe von Python die Cauchy-Gleichung

$$n(\lambda) = A \cdot \left(1 + \frac{B}{\lambda^2}\right) \quad n(\lambda) = A \cdot (1 + \lambda^2 B)$$

angepasst.

Aus der Kurvenanpassung erhält man die Materialkonstanten A und B , mit denen das verwendete Glas identifiziert werden kann.

Zusätzlich werden Messunsicherheiten sowie Fehlerfortpflanzungen berücksichtigt und grafisch dargestellt.

Verwendete Geräte:

- **Prismenspektrometer** (Hauptgerät)
bestehend aus aus:
 - o Lichtquelle (z. B. Heliumlampe oder LEDs)
 - o Kollimator (mit Spalt und Linse)
 - o Prismenstisch (zur Ausrichtung des Prismas, mit Winkelskala)
 - o Prisma
 - o Fernrohr (zum Beobachten der Spektrallinien, mit Fadenkreuz)
- **Spektrallampen** (z. B. Heliumlampe, mit bekannter Wellenlänge)
- **Leuchtdioden** (LED's, wellenlänge unbekannt)
- **Skalensystem / Nonius**(zum messen von Ablenkwinkel / Prismenwinkel)

Durchführung

3.1: Arretierung

Untersuchen, welche Teile beweglich sind, z.B. Fernrohr, Prismenstisch und Kollimator. Arretierung testen, damit sich später nichts mehr verstellt.

Bemerkungen:

3.2: Einstellung der Spaltbreite

Hier wird der Spalt am Kollimator richtig eingestellt. Er soll so schmal sein, dass die Linien scharf sind, aber noch hell genug bleiben. Ist er zu breit, werden die Linien unscharf. Ist er zu schmal, sieht man kaum noch etwas. Man sucht also einen guten Mittelweg.

Bemerkungen:

3.3: Bestimmung des Prismenwinkels γ und der Nullrichtung

Zur Bestimmung der Nullrichtung wird das Prisma entfernt und das Fernrohr direkt auf den Spalt ausgerichtet. In diesem Schritt wird zuerst der Winkel des Prismas bestimmt. Dafür misst man die Winkel der reflektierten Strahlen an den Prismenflächen. Aus diesen Werten kann man den Prismenwinkel berechnen. Danach stellt man die Nullrichtung ein, also die Richtung ohne Prisma. Diese braucht man später als Vergleich für die Ablenkung.

Messung der Winkel:

Nr.	φ_1	φ_2	γ
1	$253,5^\circ _ 4'$	$313,5^\circ _ 3'$	

Berechnung der Werte:

$$2 \cdot \gamma = \varphi_2 - \varphi_1$$

$$\gamma = |(\varphi_2 - \varphi_1) / 2|$$

Umrechnung in Grad

$$\varphi_1 (^\circ) = 253,5 \overline{6}^\circ$$

$$\varphi_2 (^\circ) = 313,55^\circ$$

$$\Rightarrow \gamma = |(253,5 \overline{6} - 313,55) / 2| = 29,991^\circ$$

Betrag, damit der Winkel für weitere Berechnungen positiv ist.

Schätzung der Messunsicherheit für γ : $1' (0,0166^\circ)$

Nullrichtung:

$$\varphi_0 = 359,525^\circ$$

Bemerkungen:

3.4.1 & 4.1.1 Messung des minimalen Ablenkungswinkels (Links)

Jetzt wird gemessen, wie stark das Licht im Prisma abgelenkt wird. Dazu dreht man das Prisma so, dass eine Spektrallinie ihre Richtung gerade ändert. Dieser Punkt ist die minimale Ablenkung. Dann wird der Winkel genau abgelesen. Das Ganze macht man für mehrere Farben und auch von der anderen Seite, um genauer zu sein. Nach den Messungen wurde die Nullrichtung erneut überprüft.

Farbe	Wellenlänge λ	$\phi_1 (^\circ \text{ } ')$	$\phi_1 (^\circ)$
Messung 1			
Dunkelrot	706,54	47° 1'	47,01666667
Rot	667,82	47° 11'	47,18333333
Gelb	587,56	47,5° 12'	47,7
Grün	501,57	48,5° 9'	48,65
Blaugrün	492,19	48,5° 17'	48,78333333
Blau	447,15	49° 7'	49,11666667
Violett	438,79	49,5° 7'	49,61666667
Messung 2			
Dunkelrot	706,54	47° 0'	47
Rot	667,82	47° 10'	47,16666667
Gelb	587,56	47,5° 15'	47,75
Grün	501,57	48,5° 9'	48,65
Blaugrün	492,19	48,5° 18'	48,8
Blau	447,15	49° 6'	49,1
Violett	438,79	49,5° 5'	49,58333333

Insgesamt

Farbe	Wellenlänge λ	$\phi_1 (^\circ)$ Durchschnitt
Dunkelrot	706,54	47,00833333
Rot	667,82	47,175
Gelb	587,56	47,725
Grün	501,57	48,65
Blaugrün	492,19	48,79166667
Blau	447,15	49,10833333
Violett	438,79	49,6

3.4.2 & 4.1.2 Messwerte (spiegelbildliche Ablenkung) (Rechts)

3.4.1, aber gespiegelt.

Farbe	Wellenlänge λ	φ_1 (_ ° _ ')	φ_1 (°)
Messung 1			
Dunkelrot	706,54	312° 22'	312,3666667
Rot	667,82	312° 9'	312,15
Gelb	587,56	311,5° 6'	311,6
Grün	501,57	310,5° 10'	310,6666667
Blaugrün	492,19	310,5° 1'	310,5166667
Blau	447,15	310° 10'	310,1666667
Violett	438,79	309,5° 10'	309,6666667
Messung 2			
Dunkelrot	706,54	312° 20'	312,3333333
Rot	667,82	312° 9'	312,15
Gelb	587,56	311,5° 6'	311,6
Grün	501,57	310,5° 10'	310,6666667
Blaugrün	492,19	310,5° 2'	310,5333333
Blau	447,15	310° 10'	310,1666667
Violett	438,79	309,5° 10'	309,6666667

Farbe	Wellenlänge λ	φ_1 (°) Durchschnitt
Dunkelrot	706,54	312,35
Rot	667,82	312,15
Gelb	587,56	311,6
Grün	501,57	310,6666667
Blaugrün	492,19	310,525
Blau	447,15	310,1666667
Violett	438,79	309,6666667

4.2 Tabelle λ & Ablenkwinkel $\delta_{\min}$

φ_1 : 253,5667

φ_2 : 313,55

$$\gamma = (\varphi_{II} - \varphi_I) / 2 = (313,55^\circ - 253,5667^\circ) / 2 = 29,9917^\circ$$

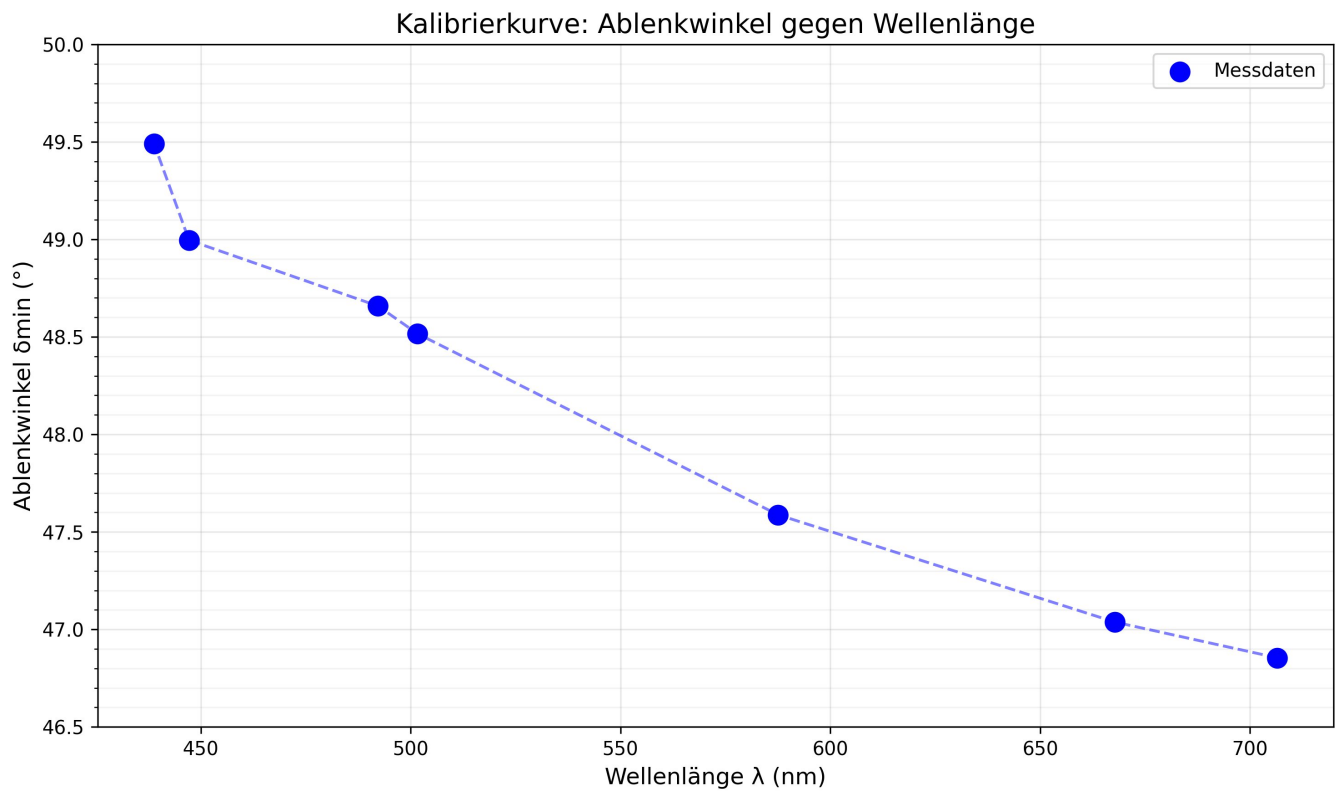
$$n_p = \frac{\sin\left(\frac{\delta_{\min} + \gamma}{2}\right)}{\sin\left(\frac{\gamma}{2}\right)}$$

$\delta_{\min}$

λ	$\delta_{\min}$	n_p
706,54	46,53333333	2,393306803
667,82	46,7	2,39771782
587,56	47,25	2,412238138
501,57	48,175	2,436533183
492,19	48,31666667	2,440240066
447,15	48,63333333	2,448512544
438,79	49,125	2,461319579

Die Werte von n können höchstens auf die 1. Nachkommastelle genau angenommen werden, da sich der angenommene Fehler von $1'$ ($0,0166^\circ$) aufsummiert.

4.3 Kalibrierkurve: $\delta_{\min}(\lambda)$



4.4.1 Dispersionskurve $n(\lambda)$ und Cauchy-Fit

$$n(\lambda) = A + B/\lambda^2 \quad (\lambda \text{ in } \mu\text{m})$$

Cauchy-Parameter (Fit-Ergebnis):

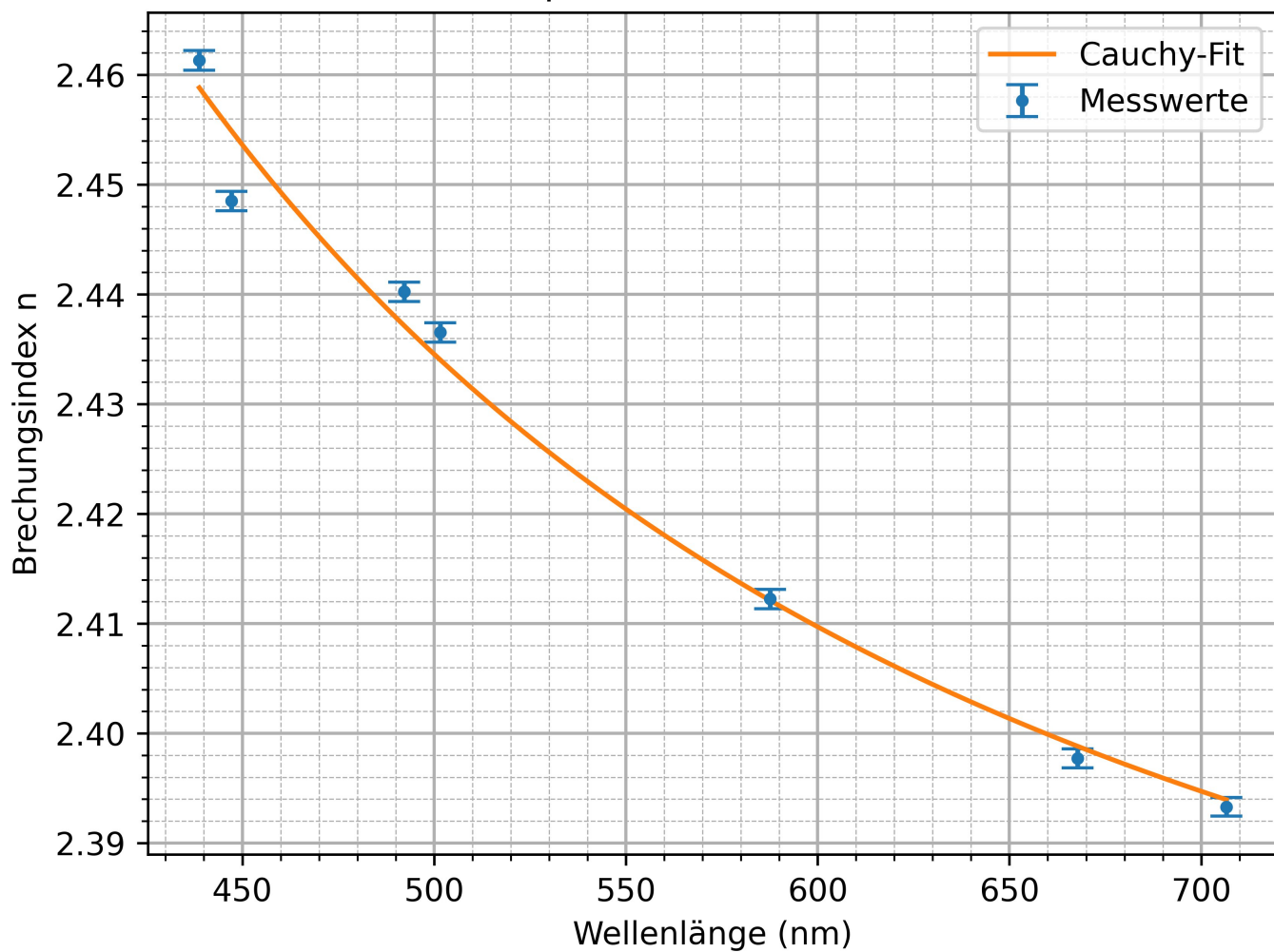
$$A = 2,353230 \pm 0.010373$$

$$B = 0,00864020 \pm 1189.54$$

Fehler der Brechungsindices:

λ (nm)	n	Δn
706.54	2.393306803	± 0.000861054
667.82	2.397717820	± 0.000862949
587.56	2.412238138	± 0.000869235
501.57	2.436533183	± 0.000879910
492.19	2.440240066	± 0.000881556
447.15	2.448512544	± 0.000885246
438.79	2.461319579	± 0.000891006

Dispersionskurve mit Fit



4.4.2 Glas Bestimmen:

$$\text{Abweichung (\%)} = 100 * \frac{|B_{\text{gemessen}} - B_{\text{tabelle}}|}{B_{\text{tabelle}}}$$

Glass	A_glass	B_glass	Abweichung (%)
Quarzglas	1,458	0,00354	144,0734463
Borsilikatglas (BK7)	1,5046	0,0042	105,7190476
Kronglas (K5)	1,522	0,00459	88,23965142
Barium-Kronglas (BaK4)	1,569	0,00531	62,71563089
Flintglas (F2)	15,904	0,00571	51,31698774
Barium-Flintglas (BaF10)	1,67	0,00743	16,28802153
dichtes Flintglas (SF10)	1,728	0,01342	35,61698957

Wie in der Tabelle zu sehen ist, ist Barium-Flintglas (BaF10) mit einer Abweichung von ca. 16,3 % das wahrscheinlichste.

4.5 Fehler der Brechungsindices

Mit Python Code, damit berechnungen schneller gehen:

Verwendete Messunsicherheiten

$$u(\gamma) = 0.0167' = 0.000278^\circ = 0.000291 \text{ rad}$$

$$u(\delta_{\min}) = 0.0167' = 0.000278^\circ = 0.000291 \text{ rad}$$

Begründung: Geschätzt aus Noniusablesung (1 Nonius-Einheit = 1')

Berechnung der cot-Werte

$$\cot(\gamma/2) = \cot(15.00^\circ) = 3.7331$$

Tabelle: Brechungsindices mit Unsicherheiten

Farbe	λ (nm)	$\delta_{\min}$ (°)	n	u(n)	u(n)/n (%)
Dunkelrot	706,54	46,8542	2,4018	0,002753	0,001146
Rot	667,82	47,0375	2,4066	0,002757	0,001146
Gelb	587,56	47,5875	2,4211	0,002771	0,001145
Grün	501,57	48,5167	2,4455	0,002795	0,001143
Blaugrün	492,19	48,6583	2,4492	0,002798	0,001143
Blau	447,15	48,9958	2,458	0,002807	0,001142
Violett	438,79	49,4917	2,4708	0,002819	0,001141

Beispielrechnung: Dunkelrot (706,54 nm)

Gegeben:

$$\gamma = 29.9917^\circ$$

$$\delta_{\min} = 46.8542^\circ$$

$$n = 2.401800$$

$$u(\gamma) = 0.000291 \text{ rad}$$

$$u(\delta) = 0.000291 \text{ rad}$$

Winkel in Bogenmaß

$$\gamma_{\text{rad}} = 29.9917^\circ \times \pi/180 = 0.523454 \text{ rad}$$

$$\delta_{\text{rad}} = 46.8542^\circ \times \pi/180 = 0.817760 \text{ rad}$$

$\cot(\gamma/2)$ berechnen

$$\begin{aligned} \cot(\gamma/2) &= \cot(14.9959^\circ) = 0.965945 / 0.258749 \\ &= 3.733132 \end{aligned}$$

$\cot((\delta+\gamma)/2)$ berechnen

$$\begin{aligned} (\delta+\gamma)/2 &= (46.8542^\circ + 29.9917^\circ) / 2 = 38.4230^\circ \\ \cot(38.4230^\circ) &= 0.783445 / 0.621462 \\ &= 1.260648 \end{aligned}$$

Formel (4) anwenden

$$u(n)/n = \sqrt{[\cot^2((\delta+\gamma)/2) \cdot u^2(\delta) + \cot^2(\gamma/2) \cdot u^2(\gamma)]}$$

$$= \sqrt{[(1.2606)^2 \times (0.000291)^2 + (3.7331)^2 \times (0.000291)^2]}$$

$$= \sqrt{[0.0000001345 + 0.0000011792]}$$

$$= \sqrt{[0.0000013137]}$$

$$= 0.001146$$

Abs. Unsicherheit

$$u(n) = n \times u(n)/n$$

$$= 2.401800 \times 0.001146$$

$$= 0.002753$$

Ergebnis: $n = 2.4018 \pm 0.0028$

Farbe	λ (nm)	n	u(n)	u(n _p)/n _p
Dunkelrot	706.54	2.4018	0.0014	0.057%
Rot	667.82	2.4066	0.0014	0.057%
Gelb	587.56	2.4211	0.0014	0.057%
Grün	501.57	2.4455	0.0014	0.057%
Blaugrün	492.19	2.4492	0.0014	0.057%
Blau	447.15	2.4580	0.0014	0.057%
Violett	438.79	2.4708	0.0014	0.057%

4.6 Gleichung 4 Herleiten

Ausgangsgleichung (2)

$$n = \frac{\sin\left(\frac{\delta + \gamma}{2}\right)}{\sin\left(\frac{\gamma}{2}\right)}$$

Partielle Ableitung:

Quotientenregel: $(u/v)' = (u' \cdot v - u \cdot v') / v^2$

Ableitung nach δ_{min} :

$$\frac{\partial n}{\partial \delta_{min}} = \frac{\cos\left(\frac{\delta_{min} + \gamma}{2}\right)}{\sin\left(\frac{\gamma}{2}\right)} \times \frac{1}{2}$$

verinfachen

$$\frac{\partial n}{\partial \delta_{min}} = \left(\frac{1}{2}\right) \times \cot\left(\frac{\delta_{min} + \gamma}{2}\right)$$

Ableitung nach γ :

$$\frac{\partial n}{\partial \gamma} = \left(\frac{1}{2}\right) \times \left[\frac{\cos\left(\frac{\delta_{min} + \gamma}{2}\right)}{\sin\left(\frac{\gamma}{2}\right)} - \sin\left(\frac{\delta_{min} + \gamma}{2}\right) \times \frac{\cos\left(\frac{\gamma}{2}\right)}{\sin^2\left(\frac{\gamma}{2}\right)} \right]$$

Verinfachen

$$\frac{\partial n}{\partial \gamma} = \left(\frac{1}{2}\right) \times \cot\left(\frac{\delta_{min} + \gamma}{2}\right) - \left(\frac{1}{2}\right) \times \cot\left(\frac{\gamma}{2}\right) \times n$$

Gauß'sche Fehlerfortpflanzung

$$u^2(n) = \left(\frac{\Delta n}{\Delta \delta}\right)^2 \cdot u^2(\delta) + \left(\frac{\Delta n}{\Delta \gamma}\right)^2 \cdot u^2(\gamma)$$

Umformen (durch n^2)

$$\frac{u^2(n)}{n^2} = \left(\frac{1}{2}\right)^2 \times \cot^2\left(\frac{\delta_{min} + \gamma}{2}\right) \times u^2(\delta_{min}) + \left(\frac{1}{2}\right)^2 \times \cot^2\left(\frac{\gamma}{2}\right) \times u^2(\gamma)$$

Ziel Formel (4):

$$\frac{u(n)}{n} = \left| \frac{1}{2} \times \cot\left(\frac{\delta_{min} + \gamma}{2}\right) \times u(\delta_{min}) \right| + \left| \frac{1}{2} \times \cot\left(\frac{\gamma}{2}\right) \times u(\gamma) \right|$$

Anhang

Alle Dateien auch zusätzlich auf GitHub:

https://github.com/FloGitHild/Physik2_Prismenspektrometer

Messwerte Protokoll

3.1: Arretierung

Untersuchen, welche Teile beweglich sind, z.B. Fernrohr, Prismentisch und Kollimator.
Arretierung testen, damit sich später nichts mehr verstellt.

Bemerkungen:

$-0,5^\circ$ (353,5° 2'
353,5° 1'

3.3: Bestimmung des Prismenwinkels γ und der Nullrichtung

Zur Bestimmung der Nullrichtung wird das Prisma entfernt und das Fernrohr direkt auf den Spalt ausgerichtet. In diesem Schritt wird zuerst der Winkel des Prismas bestimmt. Dafür misst man die Winkel der reflektierten Strahlen an den Prismenflächen. Aus diesen Werten kann man den Prismenwinkel berechnen. Danach stellt man die Nullrichtung ein, also die Richtung ohne Prisma. Diese braucht man später als Vergleich für die Ablenkung.

Messung der Winkel:

Nr.	φ_1	φ_2	γ
1	253,5° 4'	313,5° 3'	
2			
3			
4			
5			
6			
7			
8			
9			
10			

Berechnung der Werte:

$$2\gamma = \varphi_2 - \varphi_1$$

Schätzung der Messunsicherheit für γ : _____

Nullrichtung:

$\varphi_0 =$ _____

Bemerkungen:

3.4.1: Messung des minimalen Ablenkwinkels

Jetzt wird gemessen, wie stark das Licht im Prisma abgelenkt wird. Dazu dreht man das Prisma so, dass eine Spektrallinie ihre Richtung gerade ändert. Dieser Punkt ist die minimale Ablenkung. Dann wird der Winkel genau abgelesen. Das Ganze macht man für mehrere Farben und auch von der anderen Seite, um genauer zu sein. Nach den Messungen wurde die Nullrichtung erneut überprüft.

Messung 1 $\delta_{\min}$

Farbe	Wellenlänge λ	θ_1	$\delta_{\min}$
Dunkelrot Rot		51° 5' 47° 0'	48,2° 45,52°
Rot		51° 15' 47° 11'	52,8° 51° 28' 53° 4'
Rotorange gelb		52° 43' 47° 52'	52° 29' 53° 5' 53° 11'
Gelb		48,5° 9'	53° 5'
hellgrün		48,5° 17'	
hellgrün		49° 7'	
dunkelgrün		49,5° 7'	
blau			
violett			
dunkelviolett			

3.4.2: Messwerte (spiegelbildliche Ablenkung)

3.4.1, aber gespiegelt.

Messung 2 $\delta_{\min}$

Farbe	Wellenlänge λ	θ_2	$\delta_{\min}$
Dunkelrot		47,0° 0'	51,5°
Rot		47,0° 10'	51,6°
Gelb Rotorange		47,5° 15'	52°
Gelb		48,5° 9'	52,35°
hellgrün		48,5° 18'	53,5°
hellgrün		49,0° 6'	53,6°
dunkelgrün		49,5° 5'	54,0°
blau			54,4°
violett			54,5°
dunkelviolett			

Max

3.4.1: Messung des minimalen Ablenkungswinkels

Jetzt wird gemessen, wie stark das Licht im Prisma abgelenkt wird. Dazu dreht man das Prisma so, dass eine Spektrallinie ihre Richtung gerade ändert. Dieser Punkt ist die minimale Ablenkung. Dann wird der Winkel genau abgelesen. Das Ganze macht man für mehrere Farben und auch von der anderen Seite, um genauer zu sein. Nach den Messungen wurde die Nullrichtung erneut überprüft.

Farbe	Wellenlänge λ	φ_1	$\delta_{\min}$
Dunkelrot			$312^\circ 22'$
rot			$312^\circ 9'$
gelb			$311,5^\circ 6'$
grün			$310,5^\circ 10'$
blaugrün			$310,5^\circ 1'$
blau			$310^\circ 10'$
violett			$309,5^\circ 10'$

3.4.2: Messwerte (spiegelbildliche Ablenkung)

3.4.1, aber gespiegelt.

Farbe	Wellenlänge λ	φ_2	$\delta_{\min}$
D Rot			$312^\circ 20'$
Rot			$312^\circ 8'$
Gelb			$311,5^\circ 6'$
Grün			$310,5^\circ 10'$
Blaugrün			$310,5^\circ 2'$
Blau			$310^\circ 10'$
Violett			$309,5^\circ 10'$

Python: Kalibrierkurve (auch im GitHub Repository):

```
import numpy as np

import matplotlib.pyplot as plt

# 1. MESSDATEN

# Wellenlängen der He-Linien in nm

lambda_nm = np.array([706.54, 667.82, 587.56, 501.57, 492.19, 447.15, 438.79])

# Gemessene Ablenkwinkel (bereits gemittelt aus Links/Rechts)

delta_min = np.array([46.8542, 47.0375, 47.5875, 48.5167, 48.6583, 48.9958, 49.4917])

# 2. PLOT DER KALIBRIERKURVE

plt.figure(figsize=(10, 6))

# Messpunkte

plt.scatter(lambda_nm, delta_min, color='blue', s=100, zorder=5, label='Messdaten')

# Verbindungslinie

plt.plot(lambda_nm, delta_min, 'b--', alpha=0.5)

# Achsenbeschriftung

plt.xlabel("Wellenlänge  $\lambda$  (nm)", fontsize=12)

plt.ylabel("Ablenkwinkel  $\delta_{\min}$  (°)", fontsize=12)

plt.title("Kalibrierkurve: Ablenkwinkel gegen Wellenlänge", fontsize=14)

# Raster (0.5° Beschriftung, 10° Gitterlinien)

plt.grid(True, alpha=0.3, which='major')

plt.yticks(np.arange(46.5, 50.5, 0.5))

ax = plt.gca()

ax.yaxis.set_minor_locator(plt.MultipleLocator(0.1))

ax.grid(True, which='minor', alpha=0.15)

# Legende

plt.legend()

# Tight layout

plt.tight_layout()

# Speichern

plt.savefig('kalibrierkurve_delta_lambda.png', dpi=300)

print("\nBild gespeichert: kalibrierkurve_delta_lambda.png")
```

Python: Dispersionskurve & Cauchy (GitHub)

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from matplotlib.ticker import MultipleLocator

# =====
# 1. MESSWERTE
# =====

lambda_nm = np.array([706.54, 667.82, 587.56, 501.57, 492.19, 447.15, 438.79])

n = np.array([2.393306803, 2.39771782, 2.412238138, 2.436533183, 2.440240066, 2.448512544, 2.461319579])

delta_deg = np.array([46.53333333, 46.7, 47.25, 48.175, 48.31666667, 48.63333333, 49.125])

gamma_deg = 59.9833

u_delta_deg = 1/60
u_gamma_deg = 1/60

# Startwerte Fit
p0 = [1.5, 5000]

# =====
# 2. RADIANT
# =====

deg_to_rad = np.pi / 180

delta = delta_deg * deg_to_rad
gamma = gamma_deg * deg_to_rad

u_delta = u_delta_deg * deg_to_rad
u_gamma = u_gamma_deg * deg_to_rad

# =====
# 3. FEHLER BERECHNEN
# =====

term_delta = (1/np.tan((delta + gamma)/2) - 1/np.tan(gamma/2)) * u_delta
term_gamma = (1/np.tan((delta + gamma)/2)) * u_gamma

rel_error = np.sqrt(term_delta**2 + term_gamma**2)
n_error = rel_error * n

# =====
# 4. CAUCHY-FUNKTION + FIT
# =====

def cauchy(lambda_nm, A, B):
    return A * (1 + B / (lambda_nm**2))

popt, pcov = curve_fit(cauchy, lambda_nm, n, p0=p0)
A, B = popt

# =====
# 5. UNSICHERHEITEN FIT
# =====

errors = np.sqrt(np.diag(pcov))
dA, dB = errors

t = 2.365 # ca. 95% Konfidenz

A_err_95 = t * dA
B_err_95 = t * dB

```

```

# =====
# 6. PLOT
# =====

plt.figure()
ax = plt.gca()

# Raster
ax.yaxis.set_major_locator(MultipleLocator(0.01))
ax.yaxis.set_minor_locator(MultipleLocator(0.002))

ax.xaxis.set_major_locator(MultipleLocator(50))
ax.xaxis.set_minor_locator(MultipleLocator(10))

ax.grid(which='major', linestyle='-', linewidth=1)
ax.grid(which='minor', linestyle='--', linewidth=0.4)

# Messwerte
plt.errorbar(lambda_nm, n, yerr=n_error,
             fmt='o', capsize=5, markersize=3,
             label="Messwerte")

# Fit
x_plot = np.linspace(min(lambda_nm), max(lambda_nm), 1000)
plt.plot(x_plot, cauchy(x_plot, A, B), label="Cauchy-Fit")

plt.xlabel("Wellenlänge (nm)")
plt.ylabel("Brechungsindex n")
plt.title("Dispersionskurve mit Fit")
plt.legend()

plt.savefig("dispersionskurve.png", dpi=600)
plt.show()

# =====
# 7. AUSGABE MESSWERTE
# =====

print("\n--- Messwerte ---")
print("\nλ (nm)      n      Δn")

for lam, ni, ei in zip(lambda_nm, n, n_error):
    print(f"{lam:8.2f}   {ni:.9f}   ±{ei:.9f}")

# =====
# 8. FITERGEBNISSE
# =====

print("\n--- Cauchy-Fit Parameter ---")
print(f"A = {A:.6f} ± {A_err_95:.6f}")
print(f"B = {B:.2f} ± {B_err_95:.2f}")

```

Python: Fehlerfortpflanzung & Brechungsindices

```

import numpy as np

# =====
# 1. Gegebene Werte
# =====

# Prismenwinkel
gamma = 29.9917 # Grad

# Messunsicherheiten (geschätzt aus Noniusablesung)
u_gamma_grad = 1 / 60 # 1 Bogenminute in Grad
u_delta_grad = 1 / 60 # 1 Bogenminute in Grad

# Umrechnung in Bogenmaß (Radian)
u_gamma_rad = np.deg2rad(u_gamma_grad)
u_delta_rad = np.deg2rad(u_delta_grad)

print("\n" * 70)
print("AUFGABE 4.5: Fehlerfortpflanzung für n")
print("\n" * 70)

```

```
# =====
# 2. Messdaten
# =====

# Wellenlängen und Brechungsindices aus Aufgabe 4.2
lambda_nm = np.array([706.54, 667.82, 587.56, 501.57, 492.19, 447.15, 438.79])
farben = ['Dunkelrot', 'Rot', 'Gelb', 'Grün', 'Blaugrün', 'Blau', 'Violett']
n = np.array([2.4018, 2.4066, 2.4211, 2.4455, 2.4492, 2.4580, 2.4708])
delta_min = np.array([46.8542, 47.0375, 47.5875, 48.5167, 48.6583, 48.9958, 49.4917])

# =====
# 3. Fehlerfortpflanzung berechnen
# =====

# Umrechnung in Bogenmaß
gamma_rad = np.deg2rad(gamma)
delta_rad = np.deg2rad(delta_min)

# Formel (4):  $u(n)/n = \sqrt{[\cot^2((\delta+\gamma)/2) \cdot u^2(\delta) + \cot^2(\gamma/2) \cdot u^2(\gamma)]}$ 
# Berechnung von  $\cot(x) = \cos(x)/\sin(x)$ 

#  $\cot(\gamma/2)$  - konstant für alle Messungen
cot_gamma_2 = np.cos(gamma_rad / 2) / np.sin(gamma_rad / 2)

#  $\cot((\delta+\gamma)/2)$  - für jede Messung unterschiedlich
cot_delta_gamma_2 = np.cos((delta_rad + gamma_rad) / 2) / np.sin((delta_rad + gamma_rad) / 2)

# Relative Unsicherheit
rel_error = np.sqrt(
    cot_delta_gamma_2**2 * u_delta_rad**2 +
    cot_gamma_2**2 * u_gamma_rad**2
)

# Absolute Unsicherheit
u_n = n * rel_error

print(f"""
--- Berechnung der cot-Werte ---
 $\cot(\gamma/2) = \cot(\{\text{gamma}/2:2f\}^\circ) = \{\text{cot\_gamma\_2}:4f\}$ 

--- Tabelle: Brechungsindices mit Unsicherheiten ---
{farben[0]:<12} {lambda_nm[0]:>8.2f} {delta_min[0]:>10.4f} {n[0]:>12.6f} {u_n[0]:>12.6f} {rel_error[0]*100:>8.4f}%
{farben[1]:<12} {lambda_nm[1]:>8.2f} {delta_min[1]:>10.4f} {n[1]:>12.6f} {u_n[1]:>12.6f} {rel_error[1]*100:>8.4f}%
{farben[2]:<12} {lambda_nm[2]:>8.2f} {delta_min[2]:>10.4f} {n[2]:>12.6f} {u_n[2]:>12.6f} {rel_error[2]*100:>8.4f}%
{farben[3]:<12} {lambda_nm[3]:>8.2f} {delta_min[3]:>10.4f} {n[3]:>12.6f} {u_n[3]:>12.6f} {rel_error[3]*100:>8.4f}%
{farben[4]:<12} {lambda_nm[4]:>8.2f} {delta_min[4]:>10.4f} {n[4]:>12.6f} {u_n[4]:>12.6f} {rel_error[4]*100:>8.4f}%
{farben[5]:<12} {lambda_nm[5]:>8.2f} {delta_min[5]:>10.4f} {n[5]:>12.6f} {u_n[5]:>12.6f} {rel_error[5]*100:>8.4f}%
{farben[6]:<12} {lambda_nm[6]:>8.2f} {delta_min[6]:>10.4f} {n[6]:>12.6f} {u_n[6]:>12.6f} {rel_error[6]*100:>8.4f}%
""")

# =====
# 4. Beispielrechnung für eine Farbe (Dunkelrot)
# =====

print("=" * 70)
print("BEISPIELRECHNUNG: Dunkelrot (706,54 nm)")
print("=" * 70)
i = 0 # Dunkelrot

# =====
# 5. Ausgabe
# =====

print("=" * 70)
print("ZUSAMMENFASSUNG")
print("=" * 70)
print(f"""
Farbe   |  $\lambda$  (nm) | n       | u(n)    | u(n)/n (%)
-----|-----|-----|-----|-----
""")

for i in range(len(farben)):
    print(f"{farben[i]:<10} | {lambda_nm[i]:>7.2f} | {n[i]:>9.6f} | {u_n[i]:>10.6f} | {rel_error[i]*100:>8.4f}%")

print(f"""
Die relative Unsicherheit beträgt ca. {np.mean(rel_error)*100:.4f}%
Die absolute Unsicherheit beträgt ca. {np.mean(u_n):.4f}
""")
```